```
object ST {
  def apply[S,A](a: => A) = {
    lazy val memo = a
    new ST[S,A] {
      def run(s: S) = (memo, s)
    }
  }
}
```

Cache the value in case `run` is called more than once.

The `run` method is protected because an `S` represents the ability to *mutate* state, and we don't want the mutation to escape. So how do we then run an `ST` action, giving it an initial state? These are really two questions. We'll start by answering the question of how we specify the initial state.

It's not necessary that you understand every detail of the implementation of `ST`. What matters is the idea that we can use the type system to constrain the scope of mutable state.

14.2.2 An algebra of mutable references

Our first example of an application for the `ST` monad is a little language for talking about mutable references. This takes the form of a combinator library with some primitives. The language for talking about mutable memory cells should have these primitive commands:

- Allocate a new mutable cell
- Write to a mutable cell
- Read from a mutable cell

The data structure we'll use for mutable references is just a wrapper around a protected `var`:

```
sealed trait STRef[S,A] {
  protected var cell: A
  def read: ST[S,A] = ST(cell)
  def write(a: A): ST[S,Unit] = new ST[S,Unit] {
    def run(s: S) = {
      cell = a
      ((), s)
    }
  }
}

object STRef {
  def apply[S,A](a: A): ST[S, STRef[S,A]] = ST(new STRef[S,A] {
    var cell = a
  })
}
```

The methods on `STRef` to read and write the cell are pure, since they just return `ST` actions. Note that the type `S` is *not* the type of the cell that's being mutated, and we never actually use the value of type `S`. Nevertheless, in order to call `apply` and actually run one of these `ST` actions, we do need to have a value of type `S`. That value therefore